

3.1 Abstract Factory



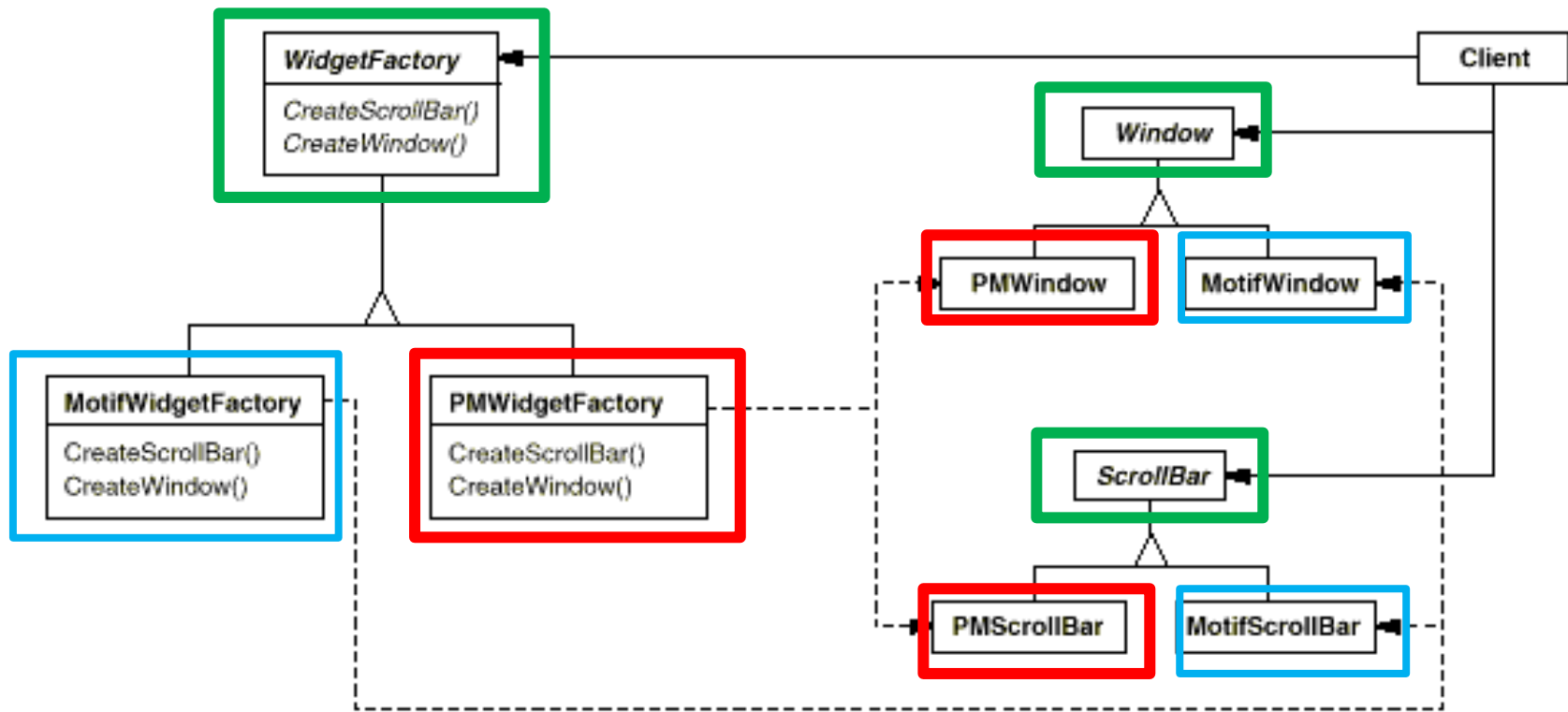
Motivation

- Consider a user interface toolkit that supports multiple look-and-feel standards, such as Motif and Presentation Manager. Different look-and-feels define different appearances and behaviors for user interface "widgets" like scroll bars, windows, and buttons.
- An application should not hard-code its widgets for a particular look and feel



Widget Factory Sample

- Structure



外部服务案例

- 比如，在一个提供给很多客户使用的产品中，我们需要使用短信服务(MessageService)和天气服务(WeatherService).
- 我们可以选择百度提供的服务，也可以选择阿里提供的服务。有些客户在商务上希望引入百度的整套服务，而另一些希望引入阿里的。
- 如何设计，使得我们的产品很容易进行在这两套服务间切换。

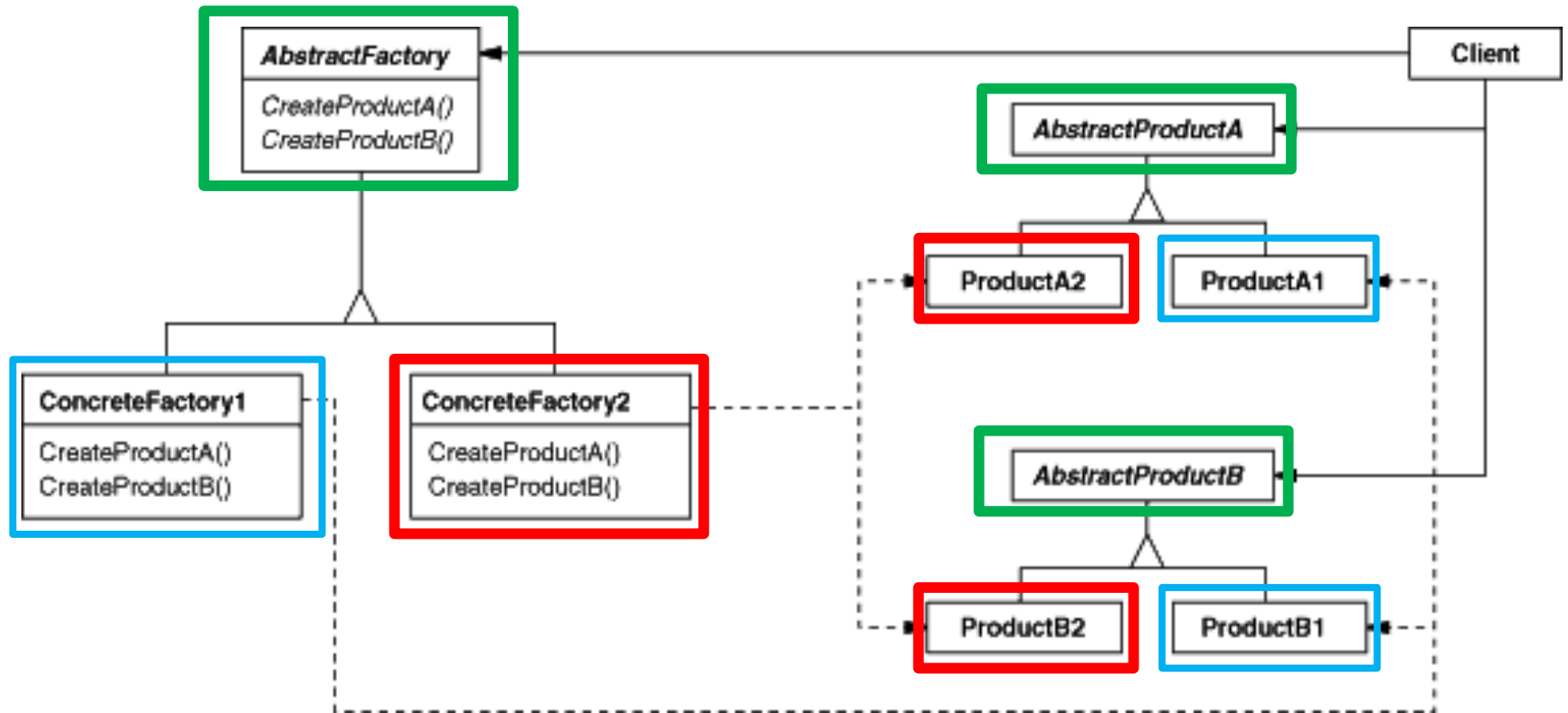


Intent

- Provide an interface for creating families of related or dependent objects without specifying their concrete classes.

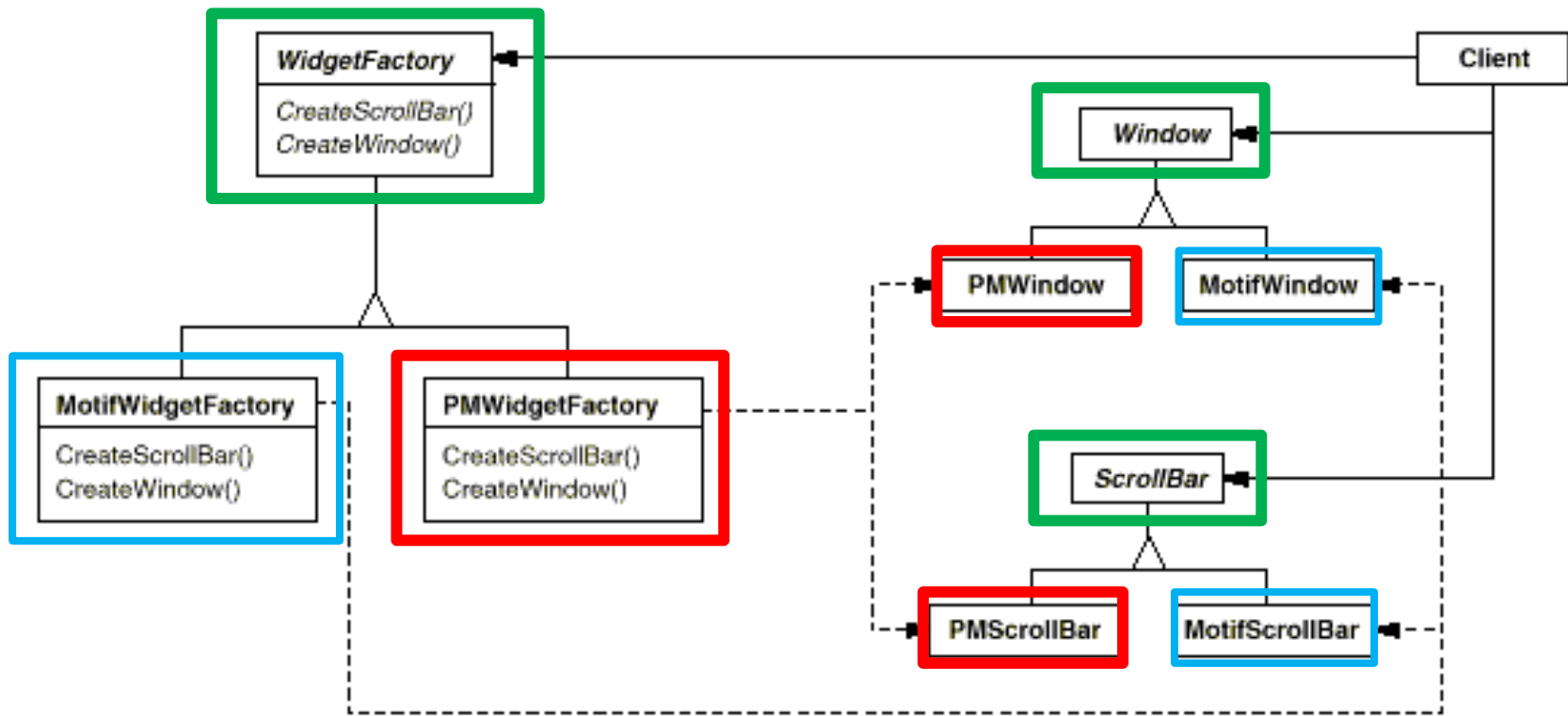


Structure



Widget Factory Sample

- Structure



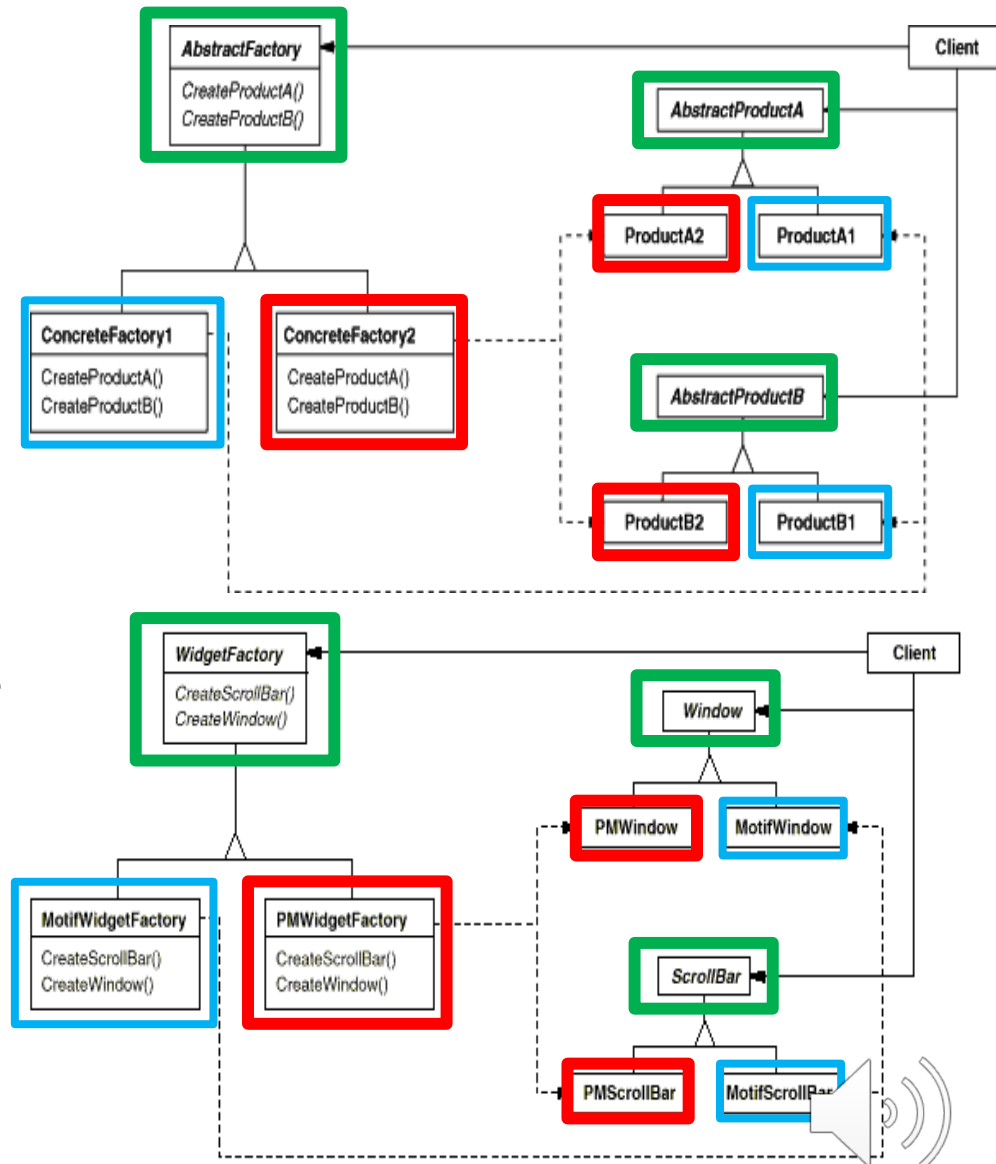
Applicability

- A system should be configured with one of multiple families of products.
- A family of related product objects is designed to be used together, and you need to enforce this constraint.
- You want to provide a class library of products, and you want to reveal just their interfaces, not their implementations.



Participants

- **AbstractFactory** (WidgetFactory)
 - declares an interface for operations that create abstract product objects.
- **ConcreteFactory** (MotifWidgetFactory, PMWidgetFactory)
 - implements the operations to create concrete product objects.
- **AbstractProduct** (Window, ScrollBar)
 - declares an interface for a type of product object.
- **ConcreteProduct** (MotifWindow, MotifScrollBar)
 - defines a product object to be created by the corresponding concrete factory.
 - implements the AbstractProduct interface.
- **Client**
 - uses only interfaces declared by AbstractFactory and AbstractProduct classes.



Consequences

- (+)It isolates concrete classes.
- (+)It makes exchanging product families easy.
- (+)It promotes consistency among products.
- (-)Supporting new kinds of products is difficult.



Sample Code

```
class MazeFactory {  
public:  
    MazeFactory();  
  
    virtual Maze* MakeMaze() const  
        { return new Maze; }  
    virtual Wall* MakeWall() const  
        { return new Wall; }  
    virtual Room* MakeRoom(int n) const  
        { return new Room(n); }  
    virtual Door* MakeDoor(Room* r1, Room* r2) const  
        { return new Door(r1, r2); }  
};
```



Sample Code

```
Maze* MazeGame::CreateMaze (MazeFactory& factory) {  
    Maze* aMaze = factory.MakeMaze();  
    Room* r1 = factory.MakeRoom(1);  
    Room* r2 = factory.MakeRoom(2);  
    Door* aDoor = factory.MakeDoor(r1, r2);  
  
    aMaze->AddRoom(r1);  
    aMaze->AddRoom(r2);  
  
    r1->SetSide(North, factory.MakeWall());  
    r1->SetSide(East, aDoor);  
    r1->SetSide(South, factory.MakeWall());  
    r1->SetSide(West, factory.MakeWall());  
  
    r2->SetSide(North, factory.MakeWall());  
    r2->SetSide(East, factory.MakeWall());  
    r2->SetSide(South, factory.MakeWall());  
    r2->SetSide(West, aDoor);  
  
    return aMaze;  
}
```



Sample Code

```
class EnchantedMazeFactory : public MazeFactory {
public:
    EnchantedMazeFactory();

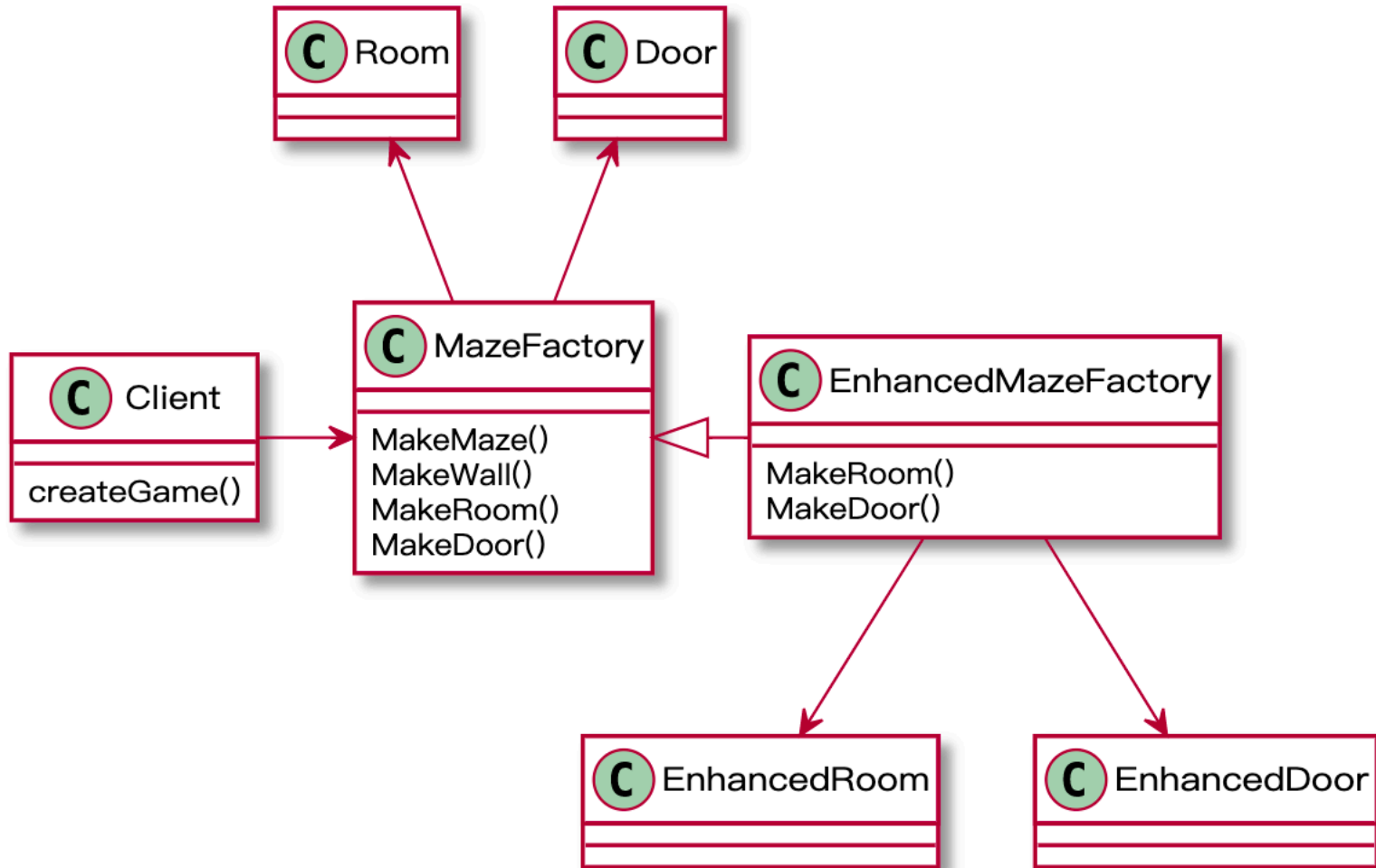
    virtual Room* MakeRoom(int n) const
        { return new EnchantedRoom(n, CastSpell()); }

    virtual Door* MakeDoor(Room* r1, Room* r2) const
        { return new DoorNeedingSpell(r1, r2); }

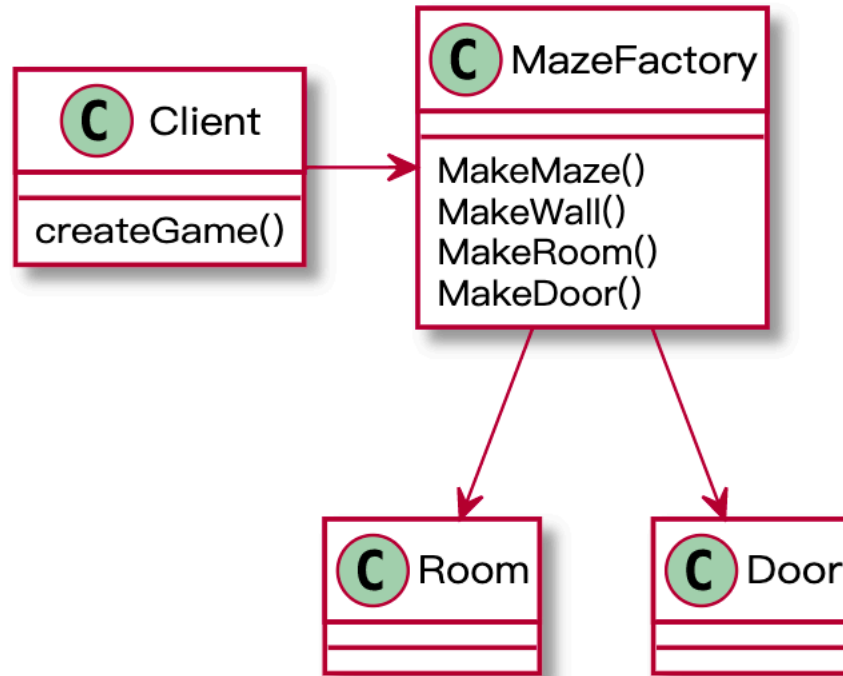
protected:
    Spell* CastSpell() const;
};
```



Sample Code



Simple Factory



抽象工厂和简单工厂的比较

- 简单工厂实现起来相对简单，但有下列缺点：
 - 开发时，不能以模块化的方式管理不同的工厂
 - 运行时，不可能同时存在两种工厂的实现
- 如果没有上述需求，使用简单工厂更加恰当。

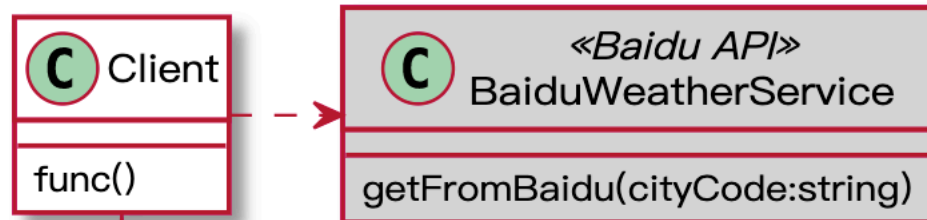


案例

- 在一个提供给很多客户使用的产品中，我们需要使用天气预报服务， 百度有天气预报服务



方案1

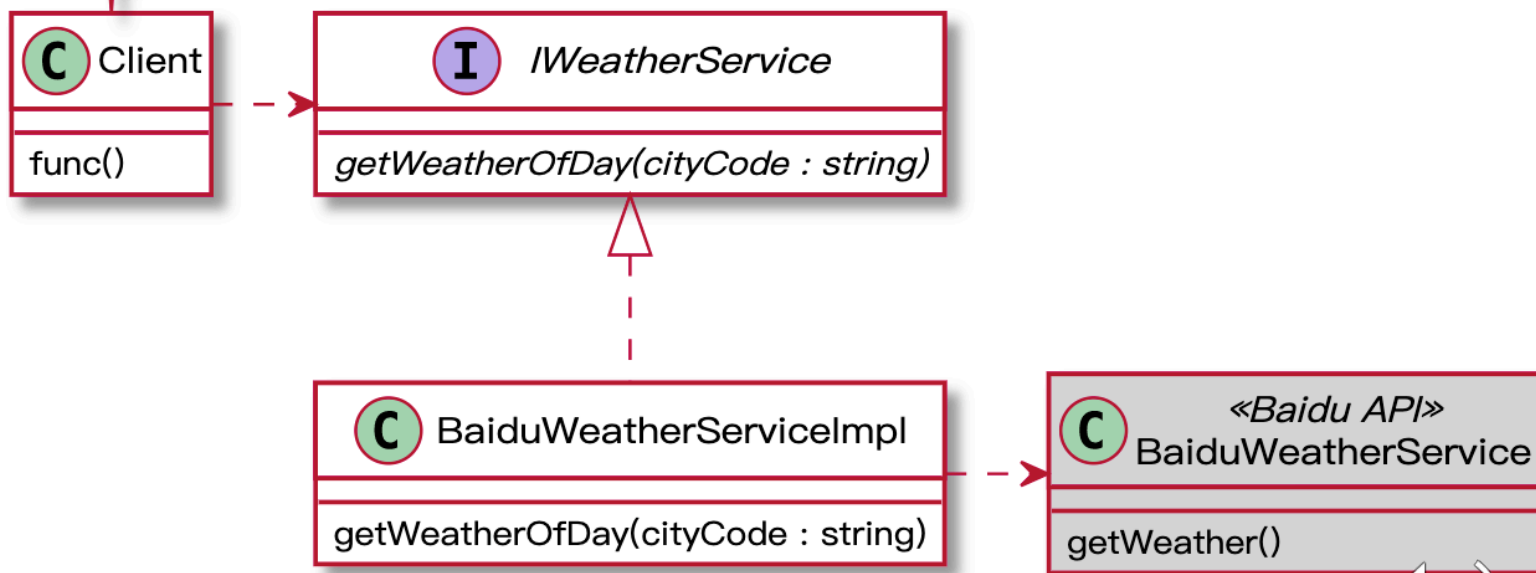


```
const ws = new BaiduWeatherService()
console.log( ws.getFromBaidu(code));
```



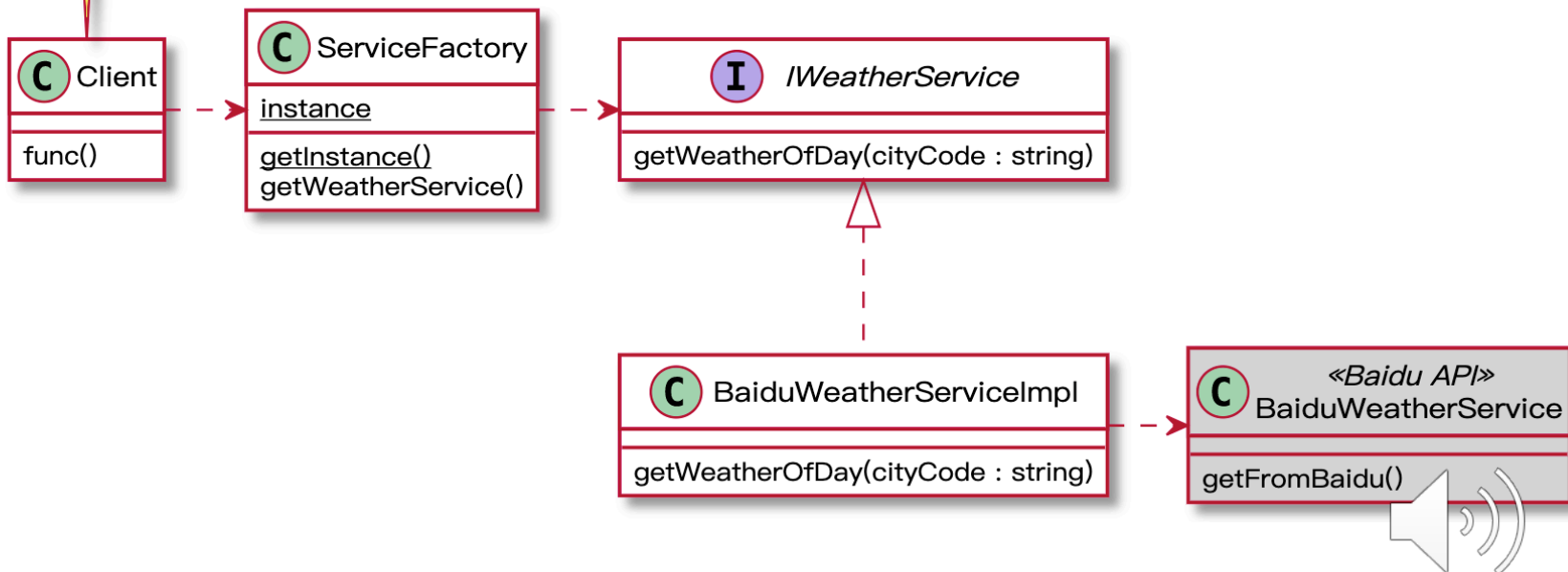
方案2

```
const ws = new BaiduWeatherServiceImpl()  
console.log(ws.getWeather(code));
```



方案3

```
const factory = ServiceFactory
.getInstance();
const ws = factory.getWeatherService();
console.log(ws.getWeatherOfCity(code));
```



方案4

